

AI Analyst — Application Documentation

MZQA AI Investment Committee — a multi-agent equity research application.

Document date: 2026-07-28 · Covers the standalone *AI_Analyst* app (backend + frontend + data layer).

1. Introduction

AI Analyst turns a natural question — “*what is this stock really worth?*” — into a structured, evidence-anchored answer produced by a **committee of LLM analysts** deliberating over a deterministic financial-analysis engine. It is carved out of the larger *MZQA Equity Terminal* and carries all the code it needs; the one thing it does not carry is the market/fundamentals data, which it reads from an existing read-only Postgres warehouse.

It offers three entry modes:

- 1. **Single stock** — the full tribunal (Advocate · Challenger · Forensic Auditor · sector specialists · Lead) plus an institutional-grade valuation and memo for one ticker.
- 2. **Group / industry** — one relative-value verdict ranking a GICS sector, industry, or screen result.
- 3. **Ideas** — a one-click Value + Sentiment scanner and a natural-language screen that surface candidates.

A fourth, quantitative surface — the **Quant** desk — adds machine-learned return/risk forecasting and portfolio optimization (see §9).

It is a research tool, not investment advice. Every number is a model output.

2. System architecture

AI_Analyst/	
- backend/	FastAPI (async, :8027) — the API + engines
- api/	routers (REST) + quant layer + async DB pool + settings
- ai_analyst/committee/	the LangGraph tribunal, valuation, evidence, group + scanner
- xbrl_sec/	data layer: DB access, LLM client, MD&A, news, macro cycle
- frontend/	Next.js 14 / React 18 / Tailwind (:3027) — the tabbed UI
- output/	generated artifacts (e.g. trained quant models)
- start_committee.bat	one-click launcher (backend + frontend)
- .env.example	configuration template

Layer	Technology	Notes
Backend	FastAPI + Uvicorn (async)	Port 8027. Blocking work (committee, qlib) runs on worker threads.
Committee engine	LangGraph (<i>StateGraph</i>)	Custom LLM-calling nodes; no external tool binding on the tribunal.
Frontend	Next.js 14, React 18, Tailwind	Port 3027, REST only. All views stay mounted (survive multi-minute runs).
Database	Postgres <i>xbrl_sec</i> , schema <i>sec</i>	Read-only. Async pool via <i>asyncpg</i> ; sync path via <i>psycopg2</i> for the engine.
LLMs	5 providers, per-request	DeepSeek (default), OpenAI, Anthropic, Moonshot, Gemini. Keys held in the browser session.

Standalone scope. The app registers a focused set of routers; the *backend/api/routers/* folder also contains a large surface inherited from the parent terminal (macro, ETF, institutional, portfolio analytics, pipeline ops, auth/billing, ...) that is **not wired into this app**. This document describes the app as it actually runs.

3. The data warehouse (*xbrl_sec*)

A pre-built, read-only Postgres warehouse (schema *sec*) is the single source of truth. The app never writes to it; a separate data pipeline populates it. Key tables:

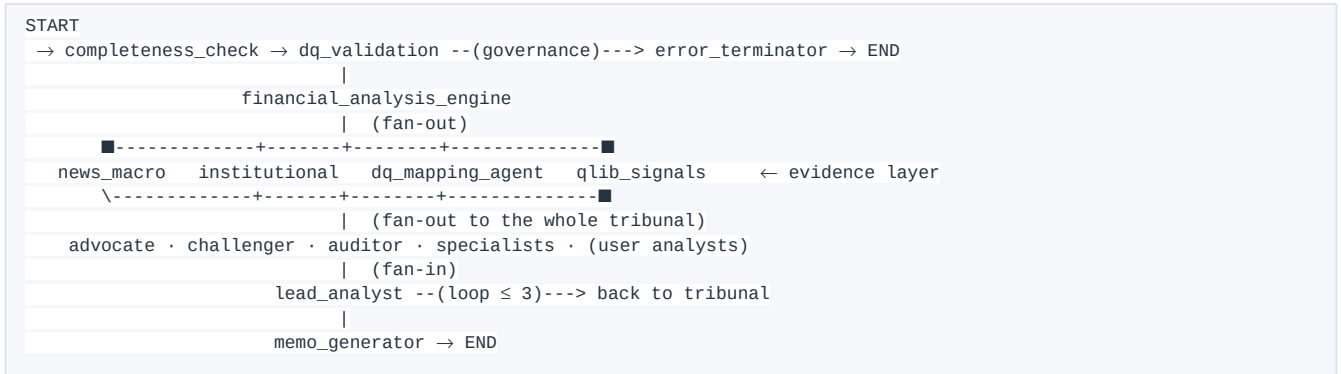
Domain	Tables	Source
Prices / OHLCV	<code>fact_prices_us</code> , <code>fact_prices_jp</code> , <code>fact_prices_etf</code>	yfinance / market feeds
Fundamentals	<code>fact_metrics_{us,jp}</code> , <code>fact_fundamentals_std_{us,jp}</code> , <code>ref_metric_definitions</code>	SEC EDGAR (US), EDINET (JP)
Factors	<code>fact_fama_french</code> , <code>fact_factor_loadings</code> , <code>fact_factor_reg_meta</code>	Fama-French + rolling regressions
Macro regime	<code>fact_cycle_state_monthly</code> , <code>fact_equity_factor_ic_regime</code>	HMM/VAE cycle model (§10)
Text / sentiment	<code>fact_mda_sections_{us,jp}</code> , news tables	Filing MD&A + news ingest
Ownership	13F institutional + insider tables	SEC 13F
Dimensions	<code>dim_company_{us,jp,intl}</code> , <code>dim_etf*</code>	GICS + identity
FX / macro	<code>fact_fx</code> , FRED/central-bank ingests	FRED, ECB, BOJ, SNB, ...

Point-in-time correctness. Fundamentals are aligned to a publication lag (`period_end + ~90 days → month end`) wherever they feed forward-looking analysis, so a signal never uses data that was not yet public.

4. The Investment Committee (core engine)

The committee is a **LangGraph state machine** (`ai_analyst/committee/graph.py`) over an `InvestmentCommitteeState` (`state.py`). It splits into two phases so several LLM providers can debate the same evidence concurrently.

4.1 Topology



- **Prepare phase** (`build_prepare_graph`): gate → engine → evidence. Deterministic and **provider-independent** — computed once and reused across providers.
- **Debate phase** (`build_debate_graph`): the tribunal → lead → memo, run per provider over a prepared state.
- `run_committee()` runs the whole pipeline; `run_prepare()` + `run_debate()` split it.

4.2 Nodes

Node	Role
<code>completeness_check</code> , <code>dq_validation</code>	Governance gate (see §4.5).
<code>financial_analysis_engine</code>	Builds the deterministic packet: <code>services.report_data_packet</code> + analytics (WACC, comps, cashflow history, incremental ROIC, reverse-DCF, prices, segments).
<code>news_macro</code>	Macro context + regime signal + scored news sentiment.

<code>institutional</code>	13F institutional ownership summary (US).
<code>dq_mapping_agent</code>	Per-ticker data-quality triage + mapping proposals (LLM).
<code>qlib_signals</code>	Machine-learned quant evidence: alpha expected return + factor risk (§9).
<code>advocate / challenger / auditor</code>	The core tribunal: the bull case, the bear stress-test, the forensic books check.
specialists + user analysts	Sector-aware and custom personas (§4.3).
<code>lead_analyst</code>	Synthesizes the debate; may loop the tribunal (≤ 3 rounds) or proceed.
<code>memo_generator</code>	Bilingual (EN/DE) investment memo.

Every analyst reads a curated evidence payload (`_agent_payload`) — canonical metrics, comps, WACC, reverse-DCF, macro regime, 13F, news, and the quant signals — and must cite the authoritative figures.

4.3 Specialist & custom analysts

Six built-in specialists (`archetypes.py`) join automatically, sector-prioritized: **Growth Extrapolator**, **Quality-of-Earnings Auditor**, **Relative-Value Arbitrageur**, **Macro-Regime Strategist**, **Sensitivity Stress-Tester**, and **Quantitative Factor Analyst** (interprets the `qlib` signals). Users can deploy their own analyst (name + mandate) from the UI; the roster persists in the browser and joins every run. Config knobs: `specialist_analyst_mode` (`auto/all/none/list`), `extra_analysts`.

4.4 Output

The committee returns (`CommitteeResponse`): a headline **fair value** and **triangulation** (SOTP-primary + DCF + multiples football field), **scenarios** (probability-weighted), **reverse-DCF** (market-implied growth), **SOTP**, full **analytics**, the **evidence bundle**, the **data-quality report**, specialist verdicts, a written **memo** (EN/DE), and a self-contained **HTML report**. There is deliberately no single BUY/HOLD/SELL label — the deliverable is a valuation with its reasoning.

4.5 Data-governance gate

Incomplete fundamentals always halt a run. An accounting-identity DQ failure is **advisory by default** (the run proceeds and surfaces a warning); the UI "Strict data-governance" toggle (`config.dq_enforce=true`) restores a hard block.

5. The valuation engine

Deterministic, transparent, and consistent on a **7-year explicit DCF horizon** (`ai_analyst/committee/`):

- **Consolidated DCF** (`dcf_engine.py`) — projected income + FCFF + bridge to per-share value.
- **SOTP** — segment-level DCF (declared primary), from off-income-statement segment disclosures.
- **Comps** (`comps.py`) — GICS peer multiples → implied value.
- **WACC** (`wacc.py`) — Fama-French-beta-derived cost of equity, segment WACC.
- **Reverse DCF** — the growth/margin the current price implies.
- **Sensitivity grid** — fair value across growth × WACC.
- **Scenario weighting** (`valuation.py`) — upside/base/downside per-share values combined into a **probability-weighted fair value**; weights can be macro-adjusted.
- **Triangulation** — blends SOTP + DCF + peer multiples into a range and an `implied_upside_pct` vs price.

6. Evidence & data-quality layer

- **News & macro** (`newsmacro.py`) — macro regime/tilt context and per-ticker scored news sentiment.
- **Institutional (13F)** (`institutional.py`) — top holders, net direction, passive share.
- **MD&A tone** — management-discussion sentiment mined from `fact_mda_sections_*`, scored on demand by an LLM.
- **Data-quality agent** — a deterministic report (`raw/standardized/metrics/recon/Yahoo` cross-check) plus an optional LLM triage that explains findings and proposes concept→variable mapping fixes (queued for review).

7. Screener & idea generation

- **Screener** (`screener.py`) — filter the US/JP/INTL universe on valuation/size/profitability/growth metrics; deterministic SQL. Also a **natural-language screen** (`/screener/ai`) that translates a prompt into filters via an LLM.
- **Value + Sentiment scanner** (`screener_agent.py`) — a one-click scan producing a 0-100 **interest score** per name. Originally a hand-weighted blend of FCF-yield / P/E / growth / MD&A tone / news; now **enhanced** with the qlib alpha model's expected return (dominant term) and **regime-conditioned factor-IC weights** (\$9–10), with the legacy formula preserved exactly as the fallback.
- **Group relative-value ranking** (`group.py`) — a **deterministic quantitative composite** (z-scored P/E, EV/EBITDA, P/B, FCF yield, growth, margins) with a per-name audit trail; an LLM adds an optional narrative.

Multi-provider behavior. The natural-language screen **fans its translation out across every keyed provider** (each surfaces a different candidate set), and the union is ranked once. The group ranking itself is deterministic; the optional narrative uses the selected provider. (Copy that previously hardcoded one provider name in offline messages was corrected so the multi-provider nature reads correctly.)

8. LLM providers & orchestration

Five providers are supported per request (`backend/llm_providers.py`), each with a fast **chat** tier (structured extraction) and a deep **reasoner** tier (narrative):

Provider	id	Dialect	Chat / Reasoner (default)
DeepSeek (default)	deepseek	openai	deepseek-v4-flash / deepseek-v4-pro
ChatGPT (OpenAI)	openai	openai	gpt-5
Claude (Anthropic)	anthropic	anthropic	claude-opus-4-8
Moonshot (Kimi)	moonshot	openai	kimi-k2.6 / kimi-k3
Gemini (Google)	gemini	openai	gemini-2.5-flash / gemini-2.5-pro

Key handling. Users paste keys under *Setup*; they live in the **browser session only** and are erased on idle/close. A server-side env key is the fallback. Three runtimes consume the registry: a sync HTTP runtime (committee engine), an async twin (routers), and a LangChain factory (structured outputs). An **offline mode** (`MZQA_COMMITTEE_DISABLE_LLM=1` or no key) still returns the deterministic analysis.

9. The quant layer (qlib)

A lean, in-process integration of Microsoft **qlib** (`api/quant/*`) adds the app's first learned forecasting:

- **Return prediction** — a cross-sectional **LightGBM alpha model** over the monthly fundamentals panel (`qlib_alpha.py`), forecasting forward returns (out-of-sample rank-IC $\approx 0.09\text{--}0.14$).
- **Risk prediction** — a factor-structured forward covariance (`qlib_risk.py`).
- **Portfolio optimization** — both a **native** SLSQP/MIP optimizer and qlib's optimizers (mean-variance, min-variance, risk-parity, inverse-vol, enhanced-indexing), **selectable at runtime** (`qlib_optimize.py`).
- **Backtest** — a cross-sectional signal backtest scored with qlib `risk_analysis` (`qlib_backtest.py`).

It surfaces three ways: the **Quant** frontend tab, the `/api/quant/*` router, and the committee's `qlib_signals` evidence node + `quant_factor_analyst`. Full detail is in the companion document **qlib_integration.pdf**.

10. The macro cycle & regime model

A macro-cycle model (HMM / VAE, `xbrl_sec/sec/cycle/`) labels each month with a market **regime** (`fact_cycle_state_monthly`). `cycle/ic.py` then computes, **conditioned on that regime**, the Spearman **information coefficient (IC)** of each fundamental factor against forward returns (`fact_equity_factor_ic_regime`) — a measure of which factors have actually predicted returns in conditions like today's.

Where it feeds forecasting today: the scanner's interest score is re-tilted by the current regime's factor-IC (via `api/quant/ic_weights.py`), so it leans on whichever factor family is currently most predictive. It is not yet a feature inside the alpha model itself (a recommended enhancement — regime-aware alpha). The committee separately receives regime context from the macro node.

11. REST API surface (active)

Base URL `http://127.0.0.1:8027`. All under `/api`.

Router	Endpoints	Purpose
<code>meta</code>	<code>GET /meta/filters, /meta/default-ticker</code>	GICS sectors/industries/exchanges.
<code>screener</code>	<code>GET /screener/meta, /screener/markets; POST /screener/run, /screener/ai</code>	Universe filtering + NL screen.
<code>screener_agent</code>	<code>POST /screener/agent/value-sentiment</code>	The Value + Sentiment scanner.
<code>ai_committee</code>	<code>POST /ai/committee, /committee/prepare, /committee/debate, /committee/iterate</code>	Single-stock committee (+ split & follow-up).
<code>ai_committee_group</code>	<code>POST /ai/committee/group</code>	Group / relative-value verdict.
<code>quant</code>	<code>GET /quant/backends; POST /quant/{alpha,risk,optimize,backtest}</code>	Quant desk (\$9).
<code>sector</code>	<code>GET /sector/returns, /sector/constituents</code>	Sector pulse.
<code>prices / kpis / company / fx</code>	<code>GET .../{ticker}, /fx</code>	Prices, KPI chips, company data, spot FX.
<code>llm_meta</code>	<code>GET /llm/providers; POST /llm/models</code>	Provider/model discovery for the UI.
—	<code>GET /api/healthz</code>	Liveness + DB check.

12. Frontend

A Next.js 14 app (`frontend/src`) with a top-nav tabbed shell (`app-shell.tsx`); all views stay mounted so a multi-minute run survives a tab switch. The API client is `lib/api.ts`; LLM keys live in a session vault (`lib/llm.ts`).

View	Purpose
Home	Landing: ticker entry, sector pulse, how-it-works.
Explore	Browse the universe / company data.
Analyze	Single-stock committee — verdict, valuation, scenarios, debate, memo, evidence.
Compare	Rank a group/sector in one deliberation.
Ideas	Value + Sentiment scanner and NL prompt screen.
Quant	The qlib desk: expected returns, factor risk, portfolio optimization (backend selector), backtest.

13. Running & configuration

Setup (once): create the backend venv and install `backend/requirements.txt`; `npm install` in `frontend/`; set `PGPASSWORD` and at least one provider key (or paste in the UI).

Run: `start_committee.bat` launches both services and opens `http://127.0.0.1:3027`. Manually:

```
# backend (from AI_Analyst/)
$env:PYTHONPATH="$PWD\backend"; python -m uvicorn api.main:app --host 127.0.0.1 --port 8027
# frontend (from AI_Analyst/frontend)
npm run dev
```

Committee CLI (token-free smoke test): `python -m ai_analyst.committee.run --ticker MSFT --offline --no-news`.

Key environment variables

Variable	Purpose
<code>DATABASE_URL, DB_SCHEMA</code>	Warehouse connection (async pool).
<code>XBRL_SEC_DATABASE_URL</code>	Sync path used by the committee engine.
<code>PGPASSWORD</code>	Postgres password (not stored in <code>.env</code>).
<code>ALLOWED_ORIGINS</code>	CORS origins for the frontend.
<code>AI_ANALYST_LLM_PROVIDER</code>	Server-default provider (else <code>deepseek</code>).
<code>{DEEPSEEK, OPENAI, ANTHROPIC, MOONSHOT, GEMINI}_API_KEY</code>	Server-side provider keys (fallback).
<code>MZQA_COMMITTEE_DISABLE_LLM</code>	Force deterministic/offline mode.
<code>QLIB_ALPHA_MODEL_DIR, QLIB_ALPHA_CACHE_TTL</code>	Quant model artifact dir + cache TTL.

14. Repository layout

Path	Contents
<code>backend/api/main.py</code>	FastAPI app factory + router registration.
<code>backend/api/routers/</code>	REST routers (active subset registered in <code>main.py</code>).
<code>backend/api/quant/</code>	qlib integration (data, alpha, risk, optimize, backtest, bridges).
<code>backend/ai_analyst/committee/</code>	The tribunal graph, nodes, state, valuation, evidence, group, scanner logic.
<code>backend/ai_analyst/{services,dcf_engine,evidence}.py</code>	Deterministic analysis engine.
<code>backend/xbrl_sec/</code>	Data layer: DB access, LLM client, MD&A/news, macro cycle model.
<code>backend/llm_providers.py</code>	The multi-provider registry.
<code>frontend/src/{app,views,components,lib}</code>	Next.js UI.
<code>docs/</code>	This documentation + <code>qlib_integration.{md,pdf}</code> .

15. Extending the app

- **Add an analyst persona** — add an entry to `committee/archetypes.py::SPECIALIST_ANALYSTS` (or deploy a custom analyst from the UI).
- **Add committee evidence** — write a node in `committee/nodes.py`, register it in `graph._add_prepare_nodes` and `_EVIDENCE_NODES`, add a state field, and surface it in `_agent_payload` (the `qlib_signals` node is the reference pattern).
- **Add a REST endpoint** — create a router under `api/routers/` and register it in `api/main.py`.
- **Add a frontend view** — add a `ViewDef` + `Tabs.Content` in `components/app-shell.tsx` and a view under `views/`.

16. Governance & disclaimer

The data-governance gate keeps analyses honest about data quality; the app is explicit when evidence (MD&A/news/fundamentals) is missing and falls back gracefully. **AI Analyst performs independent, systematic research for educational purposes. Estimates are model outputs, not investment advice.**